User Guide

Delphi App Translation Studio

Version 1.0

Last changed: August 27, 2026

Windows • Delphi VCL and FireMonkey • Win32 and Win64

Table of Contents

1. About This Guide

This guide is the complete product reference for Delphi App Translation Studio. It is written for a Delphi developer who may be new to localization, language packs, design-time components, or translation-provider APIs. It explains what each part does, why it is needed, what files are created, and what result to expect before it gives operating steps.

For the shortest first-time path, use the separate Delphi App Translation Studio Setup Wizard Guide. The Wizard Guide leads one project from preparation through translation, component setup, deployment, and final verification. This User Guide remains the authoritative reference for the Studio's full seven-page workflow, catalog editing, providers, validation, export, integration, troubleshooting, security, and file locations.

Delphi App Translation Studio is an offline-first Windows developer tool for adding localization to existing Delphi VCL and FireMonkey applications. It scans designer-authored forms and Delphi resourcestring declarations, stores the results in an editable development catalog, automatically translates unresolved text through Google Cloud Translation or DeepL, validates translation safety, and exports compact JSON language packs for the target application.

The Studio itself is built with FireMonkey, but it works with both VCL and FMX target projects. The supported build targets are Windows Win32 and Win64. macOS, iOS, Android, Linux, C++Builder, and runtime cloud translation are outside the product scope.

When the Studio starts, its landing screen offers two deliberate paths: Run Setup Wizard for a new localization setup, or Open Maintenance Studio for an existing catalog, language pack, validation, export, or integration task. This keeps first-time users out of a page whose purpose is not yet clear.

> **Offline by design.** Only the developer's Studio computer needs Internet access while creating translations. The finished target application reads local JSON language packs and never needs Internet access, a provider account, or an API key.

1.1 The Parts of the Localization System

Part	What it is	Why it exists
Translation Studio	The developer tool used while preparing a project.	Scans text, calls Google or DeepL, validates results, and builds deployable files.

Part	What it is	Why it exists
Development catalog	A detailed JSON working file under Localization\Development.	Preserves source text, translations, review state, origin, runtime classification, and locale settings between scans.
Runtime language pack	A compact JSON file under Localization\Languages, such as es-ES.json.	Supplies translated text to the finished application without Internet access or an API key.
TDAT language manager component	One nonvisual VCL or FMX component placed on the application's primary form.	Loads packs, remembers the selected language, translates open forms, and supervises forms opened later.
Language selector	A required user-facing language choice, normally the visual DAT combo-box linked to the manager.	Shows installed languages and switches immediately. A connected Language menu may be used instead.
Component integration kit	A generated, project-specific folder under the Studio export tree.	Provides component source, packs, deployment script, manifest, and exact integration instructions without rewriting the target project.

What 'manager' means in this guide. Manager is shorthand for TDATVCLLanguageManager or TDATFMXLanguageManager. It is a Delphi component installed on the DAT Localization Tool Palette page and placed once on the target application's primary form. It is not a separate program, Windows service, or online account.

1.2 What the Studio Changes - and Does Not Change

- Scanning is read-only and does not alter the selected project.
- Saving creates project-local Localization\Development catalog files.
- Export creates project-local Localization\Languages runtime packs.
- Component Integration generates a setup kit under the Studio export folder and does not write to the selected target project.
- The developer places one TDAT language manager component on the primary form in Delphi. Delphi then makes its normal, visible form-resource and unit-reference changes, exactly as it does for any other design-time component.

- Automatic Source Integration remains an explicit advanced fallback with preview, verified backup, apply, and restore.

> **Protect the original project.** Use a disposable test copy for the first evaluation. A Git repository is helpful but is not required. If Git is unavailable, copy the complete project to a separately named backup folder, confirm that the copy opens and builds, and run the Studio only against the test copy.

2. Before You Begin

The published source is currently built and release-tested with RAD Studio 13 Florence. The code is intended for modern Delphi VCL and FMX Windows projects; an older RAD Studio release may require source or package adjustments and should be treated as unverified until it passes the included build and smoke tests.

No Git installation is required to use the Studio. Git is one convenient safety check for developers who already use it; a verified folder copy or other normal backup system is equally acceptable.

Build products are placed under bin\<Platform>\<Configuration>. Compiler units are placed under dcu. The project deliberately does not target mobile or macOS platforms. After final Wizard processing, the Finish page can build and deploy selected Win32/Win64 Debug/Release combinations automatically; leave the option clear when you want to build manually.

Prepare	Reason
A clean test copy of the target project	Keeps the original application available if the evaluation is abandoned.
A successful Win32 or Win64 build	Confirms the target project is healthy before localization work begins.
Text-based DFM or FMX resources	The scanner reads designer text resources; binary legacy DFM files must first be converted through Delphi.
A Google Cloud Translation or DeepL API key	Automatic translation runs on the developer's computer. The deployed application remains offline.
RAD Studio access for one manual component step	Delphi itself installs the design package and places the manager component on the primary form.

2.1 Choose the Setup Path

Path	Choose it when	Where to begin
Setup Wizard - recommended	This is the first project, or the developer wants the Studio to lead and verify the sequence.	Select Run Setup Wizard on the landing screen and follow the separate Setup Wizard Guide.
Maintenance Studio	The project is already integrated, or the developer needs direct catalog, validation, export, provider, or integration maintenance.	Select Open Maintenance Studio on the landing screen, then choose the required page.
Manual Studio workflow	The developer understands catalogs and wants direct access to individual Project, Scan, Translate, Validation, Export, Integration, and Provider Settings pages.	Open Maintenance Studio and continue with Chapter 3 of this User Guide.
Automatic Source Integration - advanced	The component approach is unsuitable and the developer explicitly accepts reviewed source edits.	Read Chapter 11.3 and use only a protected project copy.

2.2 Build or Start the Studio

1. To use an existing build, run DelphiAppTranslationStudio.exe from bin\Win32\Release or bin\Win64\Release. Debug builds are intended for diagnosis.
2. To compile from source, open DelphiAppTranslationStudio.dproj in RAD Studio, select Win32 or Win64 and Debug or Release, then choose Project > Build DelphiAppTranslationStudio.
3. Confirm that the build writes the executable under bin\<Platform>\<Configuration> and compiler units under dcu\<Platform>\<Configuration>.
4. Start the executable and confirm that Delphi App Translation Studio opens maximized on its landing screen. The landing screen offers Run Setup Wizard for first-time use and Open Maintenance Studio for the manual workflow; the maintenance workflow rail is hidden until that choice is made.

2.3 Install the Localization Components

The Studio and the target application have different jobs. The Studio prepares the translation files. The target application needs a TDAT language manager component so it can load and apply those files at

runtime. Component installation is normally performed once per RAD Studio installation, not once per translated project.

The design package must be installed through RAD Studio's Component > Install Packages dialog. Do not use Component > Install Component, and do not try to install a .dpk source file. The correct file is the compiled Win32 Release design-time .bpl selected by the Studio.

1. If this is the first localization project, complete the Setup Wizard through its Delphi Component step. In the manual workflow, open the project and complete Scan, Translate, Validation, and Export, then open Integration and leave Component Integration selected.
2. Choose Build Integration Plan. The Studio enables Show Design BPL for the detected VCL or FMX framework.
3. Choose Show Design BPL. File Explorer selects DATLanguageManagerVCLDesign.bpl or DATLanguageManagerFMXDesign.bpl in the Studio's stable bin\packages\Win32\Release folder.
4. Start RAD Studio without opening the target form. Choose Component > Install Packages, choose Add, select the exact design .bpl shown by the Studio, and choose Open.
5. Confirm DAT Language Manager VCL design-time package or DAT Language Manager FireMonkey design-time package is listed and checked, then choose OK. Open a form and confirm DAT Localization appears in the Tool Palette.
6. Never choose Component > Install Component and never select a .dpk. A .dpk is package source; Delphi installs the compiled Win32 design .bpl through Install Packages.
7. Return to the Studio and choose Generate Component Kit. The clean kit contains ComponentSource, JSON packs, deployment support, a manifest, and README instructions; it deliberately contains no BPLs or installer scripts.
8. Normally linked applications compile the generated kit's ComponentSource units into the executable. The target does not need DAT runtime BPLs beside it.

> **Delphi-controlled installation.** The Studio does not copy BPLs into Delphi folders, write the BDS registry, or install packages automatically. RAD Studio registers the stable package from the Studio build tree through its normal Install Packages dialog. Generated per-project kits remain replaceable and are never package-registration locations.

3. The End-to-End Workflow

Step	Purpose	Primary output
1 Project	Open and identify a Delphi project.	Project profile

Step	Purpose	Primary output
2 Scan	Extract designated designer text and resourcestrings.	Scan result
3 Translate	Select a language and translate unresolved text automatically.	Saved machine-translation catalog
4 Validation	Check completeness and structural safety.	Issue list
5 Export	Create the compact offline pack.	Runtime JSON
6 Integration	Generate a non-mutating component setup kit; optionally use advanced automatic integration.	Component kit or advanced preview
7 Provider Settings	Configure and test Google Cloud Translation or DeepL.	Secure provider configuration

4. Open and Scan a Project

1. Choose Project and select Open Delphi Project.
2. Open a .dproj when available; a .dpr is also accepted.
3. Review the detected framework, platforms, form count, and source count.
4. Choose Scan. The Studio reads text DFM/FMX resources and resourcestring declarations.
5. Review the entry count, elapsed milliseconds, breakdown, and result list.

> **Binary forms.** The scanner expects text DFM/FMX resources. If a legacy DFM is binary, use Delphi's normal form conversion facilities before scanning and keep a source-control copy.

4.1 Text that is collected

- Captions, Text values, prompts, hints, headings, and designated list content.
- Memo and string-list content when it represents user-visible text.
- Delphi resourcestring symbols with stable unit-and-symbol keys.
- Multiline DFM/FMX string expressions and supported runtime UI assignments in Pascal, including Text, Caption, Header, Hint, TextPrompt, and Format templates.

- Nested utility or conversion projects are excluded when a separate DPROJ or DPR identifies them as a different application.
- Locale metadata such as date, time, number, and currency formatting is entered on the Languages page rather than guessed from UI text.

5. Create and Edit a Language Catalog

1. Choose Translate after scanning.
2. Select the source language, normally English (United States) [en-US].
3. Select the target language from the built-in language list. The Studio records its locale code and native name.
4. Review the automatically selected left-to-right or right-to-left direction.
5. Review or edit the date, time, decimal, thousands, and currency fields. Blank fields are initially populated from Delphi TFormatSettings for the target locale.
6. Choose Save.
7. Click the blue catalog path to select the saved JSON file in File Explorer.

The catalog is saved under Localization\Development using the project name and target locale. Each target language has its own development catalog. Existing translations are preserved on later scans when the stable key and source text remain unchanged. Changed source text is flagged for review, and removed entries become obsolete rather than disappearing silently.

5.1 Translation statuses

Status	Meaning
Needs translation	No usable target text is present.
AI draft	Legacy provenance retained when opening a catalog created by an earlier Studio build.
Machine translated	DeepL or Google produced a draft that requires review.
Imported	CSV supplied target text that requires review.
Edited	A person changed or entered the target text.
Reviewed	A person reviewed the target text.
Approved	The entry is release-approved.

Status	Meaning
Source changed	The source changed after a translation existed.
Excluded	The entry is intentionally omitted.
Obsolete	The source entry no longer exists in the latest scan.
Error	The entry needs correction after a failed operation or check.

5.2 Automatic Google or DeepL workflow

1. Open Provider Settings and select Google Cloud Translation or DeepL.
2. Paste the provider API key into the masked field, choose secure Windows Credential Manager storage or session-only use, and select Replace / Save Key.
3. Choose Test Connection. Resolve any authentication, billing, restriction, firewall, or quota error before translating a project.
4. Return to Translate, select the target language, and choose Translate Automatically.
5. Confirm the displayed provider and unresolved-entry count. The Studio sends only eligible unresolved source strings; Reviewed, Approved, Excluded, and Obsolete entries are not replaced.
6. The Studio first reuses approved contextual translation memory and vetted UI terminology where available. Remaining entries are sent in bounded batches. DeepL receives inferred UI context; Google Basic v2 does not support a context field, so short or ambiguous Google results are flagged for focused review. The catalog is saved automatically.

> **No extra software installation.** Automatic translation is built into the Studio. It requires only a supported provider account and API key; no command-line AI agent, Python package, browser extension, or separate translation utility is required.

5.3 Safety, context, and focused review

- Designer-property entries are applied automatically by the VCL or FMX runtime adapter.
- Pascal resourcestring entries require an explicit TranslateText(Key, Fallback) call at the intended code location. Mark Manual wiring confirmed only after adding and reviewing that call.
- Translation origin is independent of Draft, Reviewed, and Approved status. A provider result is never silently approved.
- Only Needs Translation, Source Changed, and Error entries are eligible. Existing Machine-translated, Imported, Edited, Reviewed, Approved, Excluded, and Obsolete work is protected unless the developer changes its status deliberately.

- Each scanned entry records form, component, class, property, UI role, semantic concept, contextual description, and context confidence. This distinguishes meanings such as media Play, game Play, command Schedule, the noun Schedule, weekday abbreviations, Save, Close, and media-playback timing.
- Vetted UI terminology is applied before a provider call where a supported target-language term is known, and it repairs unreviewed provider drafts when a definitive application term is available. Reviewed and Approved entries remain protected. Reviewed and Approved entries also form contextual translation memory for matching later entries.
- DeepL accepts the Studio's contextual description. Google Cloud Translation Basic v2 does not accept context or glossaries; the Studio uses local context, terminology, memory, and focused warnings around Google.
- Validation focuses on placeholders, accelerators, inconsistent repeated terms, source changes, and runtime wiring.
- Exact-source translations from other stable keys are suggestions only. The developer must explicitly accept one, and the accepted target remains Edited rather than inheriting approval.

6. Provider Settings and Secure Key Storage

Provider Settings supports DeepL API Free, DeepL API Pro, and Google Cloud Translation Basic v2. Provider accounts, billing, quotas, prices, and supported languages are controlled by the provider and may change.

A remembered key is stored as a Windows Generic Credential for the current Windows user. The key is not written to the Delphi project, JSON catalogs, exported packs, source distribution, or Git repository. Session-only keys are cleared when the Studio closes.

Control	Behavior
Provider	Select DeepL or Google Cloud Translation.
DeepL API plan	Select API Free or API Pro so the Studio uses the matching endpoint.
API key	Masked field for a new or replacement key.
Remember securely	Stores the key as a Windows Generic Credential.
Session only	Clear Remember before saving; the key is held only until shutdown.
Replace / Save Key	Records the entered key using the selected storage choice.

Control	Behavior
Test Connection	Sends a small English-to-Italian request.
Remove Key	Deletes stored and session copies for the selected provider.
Timeout / batch	Controls 5-300 second requests and 1-50 strings per request.

7. Obtain and Enter a DeepL API Key

> **Use a DeepL API account.** A normal DeepL Translator subscription is not automatically a DeepL API plan. Confirm that the account includes API Free or API Pro.

1. Open the official DeepL developer site at https://developers.deepl.com/ and review current API account requirements.
2. Create or sign in to a DeepL account and subscribe to DeepL API Free or DeepL API Pro as appropriate.
3. Open the account's API Keys tab. Create a separate key for this Studio when the account interface permits multiple keys.
4. Copy the key once and keep it out of source code, JSON catalogs, issue trackers, email, and screenshots.
5. In the Studio, choose Provider Settings and select DeepL.
6. Select API Free or API Pro to match the account. Free uses api-free.deepl.com; Pro uses api.deepl.com.
7. Paste the key into the masked field.
8. Leave Remember securely checked for Windows Credential Manager, or clear it for Use for This Session Only.
9. Choose Replace / Save Key, then Test Connection.
10. If the test fails, verify the plan selection, key status, account quota/billing, firewall, and target-language availability.

DeepL key and authentication reference: https://developers.deepl.com/docs/getting-started/auth

DeepL quickstart: https://developers.deepl.com/docs/getting-started/quickstart

DeepL multiple-key guidance: https://developers.deepl.com/docs/multiple-api-keys

8. Obtain and Enter a Google API Key

> **Google Basic v2 only.** The Studio uses Cloud Translation Basic v2 because it supports API-key authentication. Cloud Translation Advanced v3 uses different authentication and is not implemented.

1. Open Google Cloud Console at https://console.cloud.google.com/ and sign in with the account that will own billing and credentials.
2. Use the project selector to create a dedicated project, such as Delphi App Translation Studio, or select an existing project whose billing and key lifecycle you control.
3. Open Billing for that project and link an active billing account. Cloud Translation setup requires billing to be enabled even when usage might fall within a current no-charge allowance.
4. Open APIs & Services, then Library. Search for Cloud Translation API, open it, and choose Enable. Confirm that the selected project is still the intended project.
5. Open APIs & Services, then Credentials. Choose Create credentials, then API key. Create a standard API key; do not bind it to a service account for this Studio.
6. Name the key clearly, for example Delphi App Translation Studio - <computer or developer>. Google requires at least one API restriction when a key is created in the console.
7. Under API restrictions, choose Restrict key and select only Cloud Translation API (service translate.googleapis.com), then save. Do not leave the key usable with every Google API.
8. Application restrictions are recommended by Google, but the available types are websites, server IP addresses, Android, and iOS. A native Windows desktop application has no matching signed-application restriction. Use an IP-address restriction only when the developer computer exits through a stable public IP that you control; do not select website, Android, or iOS restrictions for the Studio.
9. Copy the displayed key string immediately and keep it out of source code, JSON, screenshots, email, issue trackers, and chat. The key ID or display name is not the usable key string.
10. Review Cloud Translation pricing, quotas, and billing budget alerts before a large project. API keys associate requests with the project for quota and billing.
11. In the Studio, choose Provider Settings and select Google Cloud Translation. The DeepL plan list becomes disabled because it does not apply.
12. Paste the key into the masked field. Leave Remember securely checked to store it as a Windows Generic Credential, or clear the check box for session-only use.
13. Choose Replace / Save Key, then Test Connection. A successful test translates one small English phrase to Italian.
14. If the test returns HTTP 403, verify the selected project, billing, API enablement, and API/application restrictions. HTTP 429 normally indicates a quota or rate limit. After changing a Google restriction, allow a few minutes for propagation and test again.

Google Cloud Translation setup: https://docs.cloud.google.com/translate/docs/setup

Google API-key creation and management: https://docs.cloud.google.com/docs/authentication/api-keys

Google API-key restrictions: https://docs.cloud.google.com/api-keys/docs/add-restrictions-api-keys

Cloud Translation authentication: https://docs.cloud.google.com/translate/docs/authentication

9. Direct Provider Translation

1. Open or create the target-language catalog.
2. Confirm the source and target language codes.
3. Configure and test a provider under Provider Settings.
4. Choose Translate Automatically on the Translate page.
5. Read the confirmation message showing how many unresolved strings will be sent to the provider. Cross-key suggestions are never accepted automatically.
6. Choose Yes to begin. Existing complete reviewed or approved work is preserved.
7. Wait for all batches. Transient HTTP 429 and provider server failures receive bounded retries.
8. Review every machine-translated entry for meaning, placeholders, accelerators, product terminology, tone, and available control space.
9. The Studio saves the catalog automatically. Run Validation after reviewing the results.
10. For focused work, Review and Approve the selected entry. For a large catalog, Review All and Approve All provide separate, confirmed catalog-wide decisions and save once after each operation.

> **Cost control.** The confirmation count is not a price quote. Provider billing can depend on characters and account terms. Check the provider dashboard and quotas before a large request.

10. Validate and Export

Structural validation checks catalog metadata, required translations, duplicate keys, source changes, Delphi indexed/sequential placeholders, accelerator keys, and excluded/obsolete conditions. Errors block export. Manual resourcestring wiring is reported as a runtime-readiness warning. Linguistic Reviewed and Approved states are reported separately and are not inferred from structural validation.

1. Choose Validation and Run Validation.
2. Read the summary: errors block export, warnings request review, and information messages require no action. Double-click an entry-specific issue to open that catalog entry on Translate.
3. Repeat until no errors remain.
4. Choose Export and Export Runtime Pack.
5. Record the output path under Localization\Languages.

11. Integrate a Target Application

Component Integration is the recommended path. It generates validated JSON packs, an English source pack, the applicable component/runtime source, a machine-readable manifest, deployment script, and exact setup instructions. The Studio writes these files only under export\component-integration and never opens the selected target project for writing.

The target application remains completely offline. Provider code and API keys are never added to it. One manager on the primary form supervises the application; ordinary forms do not need individual components.

1. Select Integration and leave Integration method set to Component Integration (Recommended).
2. Choose Build Integration Plan. Confirm the detected framework and translated-pack count.
3. Choose Generate Component Kit. Select README.txt and the generated manifest/files in the read-only inspection panes.
4. If DAT Localization is not already in the Tool Palette, choose Show Design BPL. It selects the stable package under the Studio's bin\packages\Win32\Release folder. In RAD Studio choose Component > Install Packages > Add, select that exact design BPL, confirm the package is listed and checked, and choose OK.
5. Open the target application's primary form in the Delphi Form Designer and place one TDATVCLLanguageManager or TDATFMXLanguageManager from DAT Localization.
6. Set ApplicationId exactly to the detected Delphi project name. Leave LanguagesFolder as Localization\Languages and set SourceLanguage to the catalog source locale, normally en-US.
7. Place TDATVCLLanguageComboBox or TDATFMXLanguageComboBox and set its LanguageManager property to the manager. A connected Language menu is the supported alternative; some visible selector is required.
8. The Setup Wizard adds the kit's ComponentSource folder to the DPROJ Base Search Path, which is inherited by Debug/Release and Win32/Win64.
9. The Setup Wizard adds a post-build command that deploys the kit's Localization folder to the active DCC_ExeOutput directory with PowerShell ExecutionPolicy Bypass.
10. Build and run the target. The saved language is applied during startup. Choosing another locale immediately retranslates open forms and saves the preference.

11.1 Component properties and behavior

Property or control	Purpose
ApplicationId	Must match runtime-pack applicationId; normally the Delphi project name.

Property or control	Purpose
LanguagesFolder	Pack folder relative to the executable unless an absolute path is supplied.
SourceLanguage	Source locale used when no translated pack is selected.
AutoLoadPreferred	Loads the saved per-user locale during manager initialization.
AutoTranslateOwner / AutoTranslateNewForms	Applies the active pack to the primary and later forms.
ReapplyOpenForms	Immediately retranslates open forms after a language change.
PreserveControlState	Protects writable user data, focus, selections, and list selection while text changes.
FormIdentityMappings	Optional FormClass=ScannerRoot mappings for renamed or inherited forms.
Language combo box	Populates from validated packs and calls the linked manager without taking over OnChange.

11.2 Runtime files and preferences

- Deploy JSON packs beside the target executable under Localization\Languages.
- Deploy-LanguagePacks.ps1 and manual copying support custom executable output layouts.
- The selected language is stored in %LOCALAPPDATA%\<ApplicationId>\language.ini.
- The executable folder can therefore remain read-only, as it normally is under Program Files.
- FMX uses additive before-show/release messages. VCL uses additive application idle/modal notifications and direct owner application.
- Selecting or reselecting a language applies the selected pack to every open form immediately. English is a real generated pack, so returning from another language restores all scanned source strings without restarting.

11.3 Automatic Source Integration (Advanced)

Use the advanced mode only when the component path is unsuitable and after committing or otherwise protecting the target project. It retains the previous generated-unit, menu-resource, preview, verified backup, atomic apply, build/deploy, restore, and Complete Reset workflow.

1. Change Integration method to Automatic Source Integration (Advanced).
2. Enter the designer language-menu component name and build the plan.
3. Generate the exact preview and inspect any file that needs closer review.
4. Authorize once, optionally select build/deploy, and choose Apply.
5. Use Restore for the session backup or Complete Reset for a recorded pre-integration baseline.

12. Translate the Studio Itself

1. Run the English Studio and select DelphiAppTranslationStudio.dproj.
2. Scan it and create the desired language catalog.
3. Translate, review, validate, and export the pack.
4. Copy the exported pack to the Studio project or deployed executable's Localization\Languages folder.
5. Add the locale to the designer-authored Studio Language menu when a selectable menu item is required.
6. Close the running Studio, rebuild when the menu changed, and start the executable.
7. Select the new language. Open forms change immediately and the preference is retained for the next launch.

The running executable is never overwritten. The JSON pack and preference are separate from the executable, and the rebuild supplies the updated menu on the next run.

13. Troubleshooting

Symptom	Likely action
Project framework unknown	Open the .dproj, confirm VCL/FMX units and project metadata, then rescan.
No scan entries	Confirm text DFM/FMX resources and designated user-visible properties.
HTTP 401/403	Replace the key; verify provider plan, API enablement, restrictions, and billing.
HTTP 429	Quota or rate limit reached; wait, reduce batch size, or review provider quota.

Symptom	Likely action
DeepL test fails only on one plan	Select API Free or API Pro to match the account endpoint.
Google test fails	Confirm Basic v2 Cloud Translation API is enabled and key API restriction permits it.
Export blocked	Run Validation and correct every error.
Language selector is empty	Confirm valid nonempty packs, matching applicationId, and the deployed Localization\Languages folder; call RefreshLanguages after late deployment.
Component missing from Tool Palette	Use Show Design BPL, then Delphi Component > Install Packages > Add. Select the exact self-contained Win32 design BPL.
Preference cannot be found	Check %LOCALAPPDATA%\<ApplicationId>\language.ini, not the executable folder.
Target remains English	Confirm manager ApplicationId, JSON applicationId/locale, deployment folder, preference, and manager initialization errors.

14. Security and Privacy

- Only confirmed source strings are sent to the selected provider.
- Do not send secrets, personal data, customer data, or regulated information as UI text.
- Remembered keys are Windows Generic Credentials; session keys are not persisted.
- Removing a key affects only the selected provider.
- Provider settings JSON contains no API key.
- Catalogs, runtime packs, deployment packages, logs, backups, and Git should contain no key.
- Rotate a key immediately in the provider console if it may have been exposed, then replace it in the Studio.

15. File and Folder Reference

Location	Contents
Localization\Development	Editable full development catalogs.
Localization\Languages	Compact offline runtime JSON packs.
export	Generated previews and temporary product output.
export\component-integration	Recommended non-mutating component setup kits.
packages\runtime / packages\design	Core/framework runtime packages and VCL/FMX IDE packages.
docs\guides / docs\pdf	Editable guides and companion PDFs.
%LOCALAPPDATA%\DelphiAppTranslationStudio	Studio settings and language preference, but no remembered key.
Windows Credential Manager	Remembered DeepL/Google Generic Credentials.
%LOCALAPPDATA%\<ApplicationId>	Integrated application's selected-language preference.

16. Provider Reference and Change Notice

Provider setup screens and commercial terms can change after this guide is published. Before creating a production key, compare these steps with the official pages listed in Chapters 7 and 8. Prefer a dedicated, restricted key and review the provider dashboard after the first bulk run.

This guide documents the implemented Studio as of August 27, 2026.

17. Stabilization and Recovery

The stabilization release treats a language change, scan, save, export, component-kit generation, and deployment as bounded operations. The Studio keeps the interface responsive during long scans and provider work, exposes cancellation at safe boundaries, and reports a concise diagnostic instead of repeatedly processing UI messages inside the operation.

Catalogs, preferences, provider settings, runtime packs, layout overrides, glossaries, translation memories, and interchange files use atomic replacement. When a previous valid file exists, a recoverable .previous copy is retained. An invalid current file is quarantined instead of being silently accepted or overwriting the last valid state.

17.1 Strict runtime-pack admission

- A pack must have unique JSON keys and a file name that matches its locale.
- applicationId, framework, format version, locale, and checksum must match the target runtime contract.
- A translated pack is accepted only with a compatible source-language pack; the source pack supplies the deterministic fallback and restores the source language after switching.
- An incompatible saved language preference is logged, replaced with the validated source language, and does not prevent application startup.

17.2 VCL and FMX form lifecycle

FMX applies the active language through its additive before-show lifecycle. VCL translates the manager owner during initialization, retranslates open forms when the language changes, and discovers later forms through additive application notifications. Delphi VCL does not expose one universal additive before-show event for every dynamically created modeless form. When first-paint translation is mandatory for such a form, call the manager's ApplyToForm method immediately before Show. Call ApplyToForm before ShowModal for the same explicit guarantee on modal forms.

Do not add a timer, idle replay, or repeated post-show translation loop to compensate for application-specific lifecycle code. Such replay can make the UI sluggish and can overwrite live control state.

17.3 Layout, RTL, HTML, and diagnostics

- RTL/LTR state is reapplied from the immutable designer baseline whenever the language changes; direction does not accumulate across switches. Inactive FMX tab pages resolve their layout from the live tab client or its saved designer width rather than from temporary 8- or 50-pixel framework placeholders.

- Automatic fitting is language-neutral and measurement-driven. Buttons retain their designer width when the caption already fits, then grow only into verified free space, reduce the font only to the readable floor, and wrap only as a final fallback. Headings, labels, check boxes, grids, and cards use the same available-space contract in every language.
- Cards and other designer-authored containers retain outer gutters and at least 12 pixels below their visible content. Container growth uses a fixed number of passes and never starts a layout replay loop.
- Automatic layout acceptance is limited to relative, reversible changes. Absolute geometry and grid-column widths remain review decisions. Grid headings may reduce font size to fit, and reviewed column plans balance the available width without language-specific exceptions.
- HTML localization changes exact visible prose while preserving comments, scripts, styles, protected nodes, product identifiers, technical tokens, and markup structure. The browser contract also applies the pack direction and safe wrapping/padding rules to the document.
- Scanner output distinguishes identical duplicate observations from conflicting duplicate keys; conflicts remain visible for correction.
- The diagnostic log records recovery, rejected packs, fallback decisions, deployment rollback, and cancellation without recording provider credentials.

17.4 Language selection, progress, and external text

- A new project does not silently default to German or any other target language. Select the intended locale explicitly. Reopening a development catalog restores its saved source language, target locale, native name, direction, and locale formats without firing a change event that overwrites them.
- Arabic, Hebrew, Persian, Urdu, Pashto, and any locale Windows identifies as RTL use the same locale-facts and runtime-direction path. Language-specific vocabulary repairs are restricted to exact semantic states, such as On and Off, and never substitute for geometry rules.
- Long scans and provider translations show a blocking operation card with the current stage, progress, and a safe Cancel action. File > Exit provides the normal application exit path. During the mandatory ZIP backup, cancellation is accepted immediately by the interface and completed after the current archive call returns.
- The scan summary separates form properties, Delphi resourcestrings, runtime UI assignments, form files, and source files. A larger count therefore has an auditable source instead of appearing as one unexplained total.
- Operator-facing prose stored in project JSON is scanned only when the project root contains dat-translatable-resources.json. The manifest names a project-contained directory, a file-name pattern, and the exact JSON property names that carry visible prose. Arbitrary JSON is never scanned because it may contain identifiers, settings, or user data.